

Kuhn Poker על Policy-Space Response Oracles

נדב פיירמן

1. הבעיה והשאלה

בלמידת חיזוקים תחרותית מאמנים בדרך כלל סוכן שמשחק נגד עצמו ולומד מהתוצאות. זה נקרא self-play, ויש לו חולשה.

יש משחקים שבהם אין דרך משחק אחת שמנצחת את כולן, כמו באבן-נייר-ומספריים. בכל פעם שהסוכן לומד לנצח את הגרסה הקודמת של עצמו הוא מאבד את מה שידע לנצח לפניה. הוא מסתובב במעגל ומשתפר מול היריב של הרגע במקום מול המשחק.

PSRO מחזיק אוסף מדיניות לכל שחקן במקום אחת. בכל סבב הוא בודק מי מנצח את מי בין כל הזוגות, מחשב באיזו תדירות כדאי לשחק כל מדיניות, מאמן מדיניות חדשה נגד התערובת ומוסיף אותה. הלמידה עוברת מרמת הפעולה לרמת המדיניות: לא איזו פעולה טובה יותר, אלא איך לשלב בין דרכי משחק שלמות. השאלה: האם האוסף מייצר שחקן עמיד יותר מ-self-play באותו תקציב אימון? המדד המרכזי הוא exploitability — כמה יריב ירוויח נגדנו אם הוא יודע בדיוק איך אנחנו משחקים. אפס אומר שאין מה לנצל.

2. המשחק והתגמולים

Kuhn Poker הוא פוקר מוקטן. שלושה קלפים, J קטן מ-Q קטן מ-K, כל שחקן מכניס אסימון ומקבל קלף אחד. שתי פעולות לכל שחקן: לא להמר, או לקפל אם היריב המר; ולהמר אסימון, או לשלם על הימור. מי שמקפל מפסיד אסימון, בהשוואת קלפים בלי הימור הזוכה מקבל אסימון, ואם ההימור נענה הוא מקבל שניים. שחקן רואה רק את הקלף שלו ואת מה שקרה עד כה, וזה נותן לו שש מצבים להחליט בהם, כלומר 64 דרכי משחק. אפשר לעבור על כולן, ולכן ה-exploitability כאן מחושב במדויק ולא מוערך. זו הסיבה לבחירה במשחק הקטן.

יש לו גם פתרון מתמטי ידוע. ערך המשחק לשחקן הראשון הוא $1/18$, ויש לו משפחה של פתרונות נכונים התלויה בפרמטר אחד: α , ההסתברות לבלף עם J, בכל ערך בין 0 ל- $1/3$; אז ההסתברות לשלם עם Q היא $\alpha + 1/3$ ולהמר עם K היא $3 \cdot \alpha$. לשחקן השני פתרון יחיד, בהסתברות $1/3$. OpenSpiel מפרסמת שתי אמות מידה נוספות, $11/12$ למדיניות אחידה ו-2 למדיניות שמקפלת תמיד, ושתיהן מתקבלות כאן בדיוק.

סוכני הבסיס: random, tight שמהמר רק עם K, aggro שמהמר תמיד, ולומד self-play. במפגשים ביניהם self-play ו-aggro מנצחים את random, אך אף אחד לא מנצח את tight ולא את הפתרון המתמטי. אין דרך משחק אחת שמנצחת את כולן, וזה התנאי שהמפרט דורש. לומד ה-self-play מתכנס למדיניות שמהמרת תמיד וחוזר על מחזור באורך 4.

3. ארכיטקטורת PSRO

תשעה מודולים, כל אחד נשען רק על אלה שלפניו. הכול נכתב מאפס ב-NumPy, SciPy ו-pandas, בלי ספריית למידת חיזוקים.

קובץ	שורות	תפקיד
game.py	699	עץ המשחק, מצבי ההחלטה, מדיניות, הרצת משחקים
exact.py	352	החישוב המדויק: ערך, תגובה טובה ביותר, exploitability
metagame.py	291	טבלת מי-מנצח-את-מי: אומדן, שגיאת תקן, רווחי סמך
oracles.py	448	שלושת המאמנים, ולומד self-play
metasolvers.py	338	שמונה דרכים לחשב את התערובת
psro.py	458	הלולאה, הפרמטרים, שמירה והמשך
analysis.py	734	מדדי גיוון, סטטיסטיקה, כל הגרפים
experiments.py	588	חמשת הניסויים והאבלציות, שורת פקודה
test_psro.py	649	89 בדיקות, כולן עוברות

הלולאה מבצעת בכל סבב את ששת השלבים שהמפרט מונה ורושמת exploitability, אנטרופיה, ערך, תמיכה, אפיזודות וזמן. כל הפרמטרים הנדרשים יושבים באובייקט אחד: סבבים, תקציב אימון, אפיזודות הערכה, פותר, מאמן, גודל אוכלוסייה, seed, סף עצירה וגיוון. כל מדיניות, טבלה ותערובת נשמרות, וריצה שנקטעה ממשיכה מהמקום שנעצרה.

אם מחליפים את המאמן בחישוב מדויק ואת הטבלה המשוערת במדויקת, הלולאה הופכת ל-double oracle הקלאסי: exploitability של $4.16e-17$ בשבעה סבבים, ושני החסמים על ערך המשחק נסגרים על $1/18$. זו בדיקה שהאלגוריתם מבצע על עצמו, בלי הפתרון החיצוני.

4. המאמן שמייצר תגובה

הרכיב שמאמן מדיניות חדשה נגד תערובת היריב נקרא response oracle. מומשו שלושה מאפס: Q-learning, טבלאי עם חקירה אקראית וקצב למידה שיורד לפי מספר הביקורים; REINFORCE, שמייצר מדיניות הסתברותית; ומעבר על כל 64 דרכי המשחק, שאינו למידה אלא הגבול שאף מאמן לא יעבור. בכל אפיזודת אימון מוגרל יריב אחד מהתערובת והוא קבוע עד סופה, כנדרש.

עם קצב למידה קבוע האימון נתקע: מי שלא מהמר עם J מפסיד בדיוק אסימון אחד בלי שונות, ומי שמהמר מפסיד 0.5 בממוצע אבל התוצאה נעה בין מינוס שניים לפלוס אחד. רצף מזל רע מפיל את ההערכה הרועשת מתחת לוודאית, הסוכן נוטש והחקירה מתמוטטת. קצב שיורד לפי מספר הביקורים פתר את זה. באימות Q-learning מגיע לתגובה הטובה ביותר בפער אפס בכל עשרת המקרים, כבר ב-2,000 אפיזודות. REINFORCE יורד מפער 0.013 ל-0.0007 בתקציב גדול פי עשרה. מול הפתרון המתמטי שניהם מקבלים בדיוק $1/18$.

5. חישוב התערובת

אחרי שיש טבלת מי-מנצח-את-מי צריך להחליט באיזו תדירות לשחק כל מדיניות. המפרט דורש שלוש דרכים, מומשו שמונה: חלוקה אחידה, המדיניות האחרונה בלבד, fictitious play, נאש בתכנון ליניארי, רפליקטור, רפליקטור עם רצפת הסתברות, rectified Nash ו-regret matching. הבחירה הזאת היא בעצם בחירת האלגוריתם: המדיניות האחרונה נותנת self-play, ונאש נותן double oracle. כל השמונה נבדקו על ארבעה משחקים שפתרונם ידוע, ביניהם משחק לא-סימטרי שפתרונם תערובת לא אחידה בהסתברויות $1/3, 1/2$ ו- $1/6$ — הוא נבחר כדי שמימוש שמחזיר תמיד חלוקה אחידה ייכשל בו.

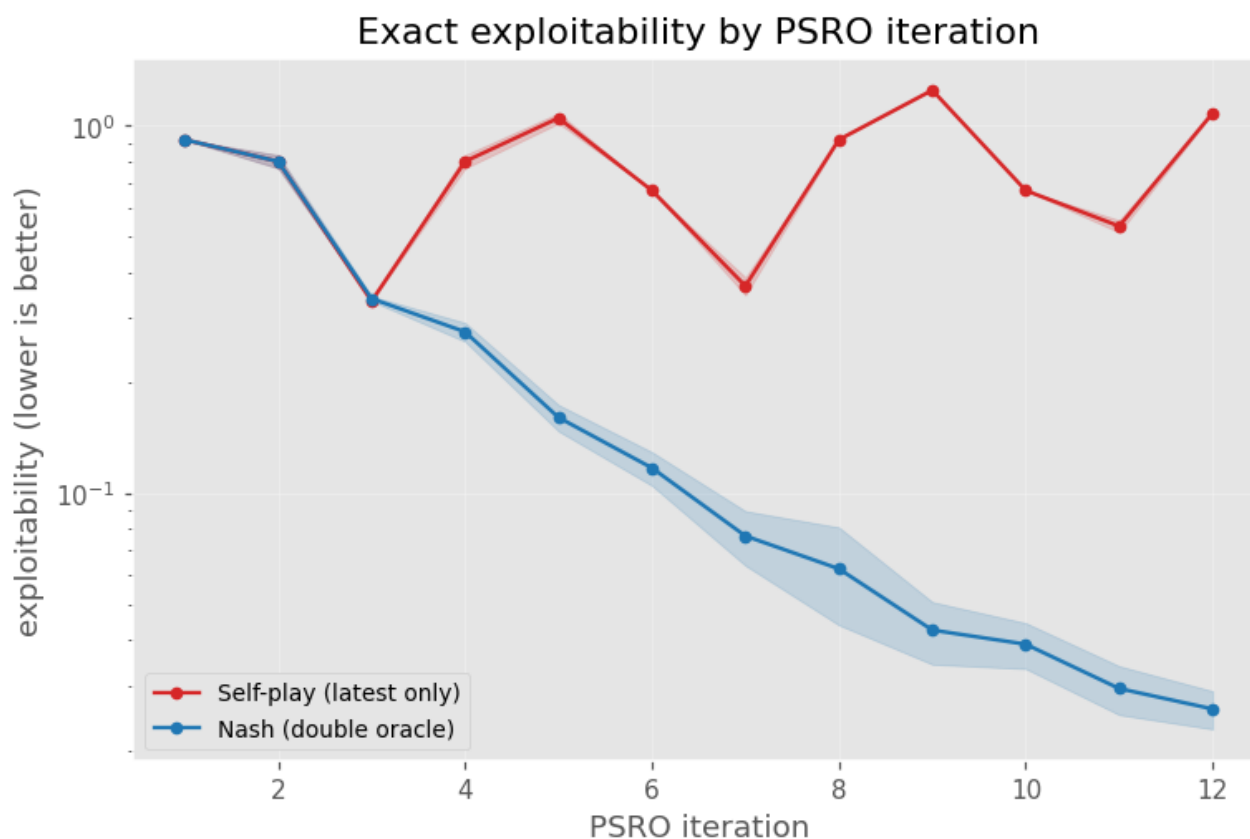
6. מתודולוגיה ורווחי סמך

כל תא בטבלה נאמד מחמישה בלוקים בלתי תלויים של 400 משחקים, ושגיאת התקן מחושבת על ממוצעי הבלוקים. רווח הסמך נבנה בהתפלגות t עם מכפיל 2.776 ולא 1.96, כי חמישה בלוקים הם מדגם קטן. האומדן נבדק מול הטבלה המדויקת: 24 מתוך 25 התאים מכילים את הערך האמיתי, השגיאה הגדולה 0.0875, בלי הטיה לכיוון אחד. הוספת מדיניות מחשבת חמישה תאים במקום שלושים. כל ניסוי רץ על חמישה seeds. בניסוי 1 נוסף מבחן Mann-Whitney, כי מבחן t מחלק באפס במדדים שבהם ל-self-play אין שונות כלל, וגם דיווח בסגנון reliable: ממוצע בין-רבעוני, רווח בוטסטרפ והסתברות השיפור. תקציב הריצה: 12 סבבים, 5,000 אפיזודות אימון לכל שחקן בסבב ו-10,000 יחד, 2,000 אפיזודות הערכה לתא, ואוכלוסייה שמגיעה ל-13. כל 131 הריצות נשמרות עם קובץ פרמטרים, והרצה מלאה מאפס ארכה 23.8 דקות. ה-exploitability כאן הוא סכום שני הפערים כהגדרת המפרט, ו-OpenSpiel מדווח חצי מזה.

7. תוצאות

7.1 ניסוי 1 — self-play מול PSRO

PSRO מגיע ל-0.0315 מול 1.2500 ב-self-play, פי 39.7, בלי חפיפה בין רווחי הסמך. שגיאת ערך המשחק חדה עוד יותר: 0.0004 מול 0.8056. כלומר self-play לא רק ניתן לניצול, הוא גם לא מגיע לערך הנכון של המשחק, כי הוא מווד את עצמו מול עצמו. המחזוריות נמדדת ישירות: מול גרסאות מוקדמות של עצמו self-play מפסיד -0.4167 לעומת -0.0326 ב-PSRO. האנטרופיה מסבירה למה — 1.17 מול 0.00, כלומר PSRO מגריל בין כמה מדיניות ו-self-play נעול על אחת. מבחן Mann-Whitney נותן p של 0.012 או פחות בחמישה מששת המדדים, ובשניים מהם 0.0075 — הרצפה האפשרית בחמישה מול חמישה. מדד אחד הפוך, ונשאר בכוונה: מול יריבים חלשים שלא נראו self-play מרוויח יותר, 0.0965 מול 0.0538. זו אינה סתירה אלא ההבחנה שבלב העבודה — מדיניות שיווי משקל לא מנסה לנצל טעויות, היא מבטיחה שלא ינצלו אותה.



איור 1. exploitability לפי סבב, ממוצע על חמישה seeds עם רצועת שגיאת תקן. נמוך יותר הוא טוב יותר.

7.2 ניסוי 2 — השוואת שמונת הפותרים

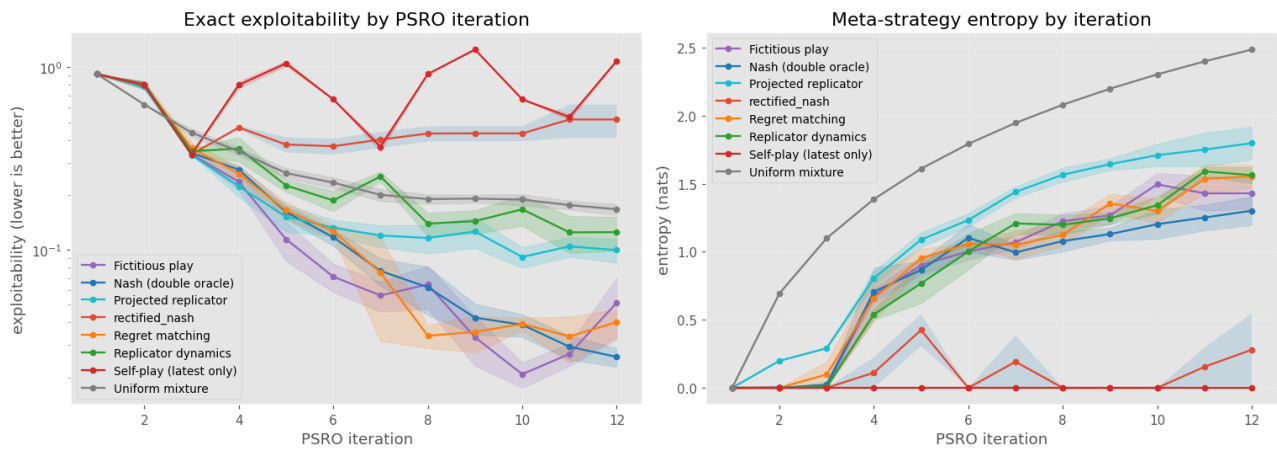
פותר	exploitability	רווח 95%	תמיכה	אנטרופיה
fictitious_play	0.0282	0.0118	8.6	1.4257
nash	0.0315	0.0150	4.4	1.1686
regret_matching	0.0516	0.0391	13.0	1.5189
replicator	0.0947	0.0674	10.4	1.4990
projected_replicator	0.1107	0.0285	13.0	1.8505
uniform	0.1462	0.0469	13.0	2.5649
rectified_nash	0.5178	0.2991	1.8	0.2459
self_play	1.2500	0.0000	1.0	0.0000

fictitious_play ראשון, אבל רווחי הסמך שלו ושל nash חופפים כמעט לגמרי וגם regret_matching בטווח:

שלושת הראשונים אינם ניתנים להפרדה בחמישה seeds. ההסבר לסדר — fictitious_play מייצר תערובת רכה שפחות נפגעת מרעש בטבלה, ואילו nash מנצל את הטבלה במדויק ולכן גם את השגיאות שבה. אנטרופיה גבוהה אינה מטרה: חלוקה אחידה מגיעה ל-2.56 והיא רק שיטית.

הממצא החד ביותר בא ממדד שאינו תלוי בפותר: אצל חמישה מתוך שמונה האוכלוסייה עצמה כבר מכילה פתרון מלא של המשחק, וכל ה-exploitability שנשאר הוא מחיר בחירת התערובת ולא מחיר האיסוף. רק חלוקה

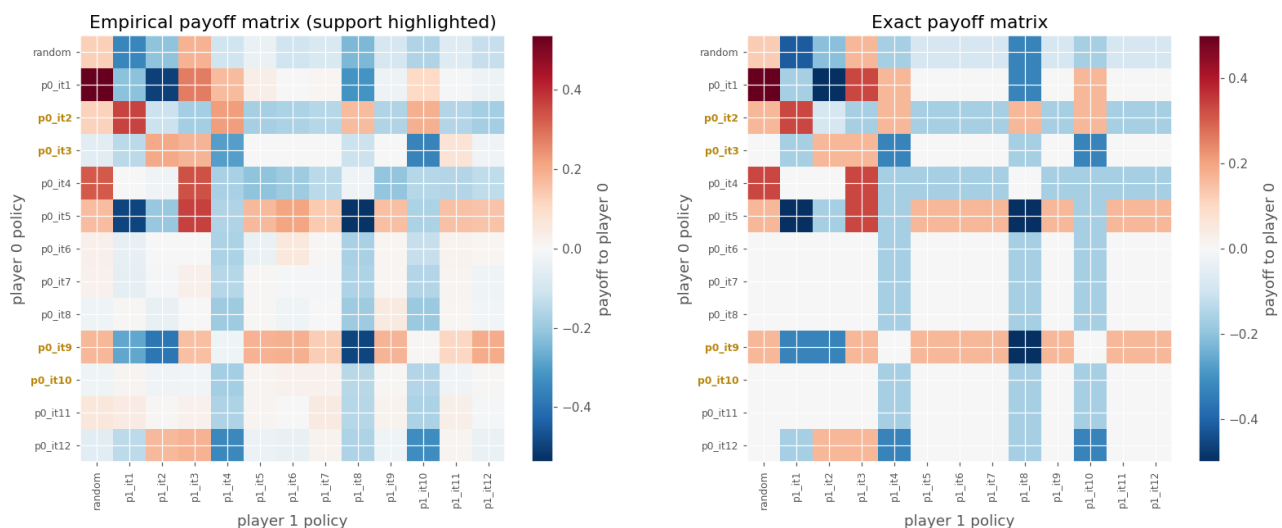
אחידה, rectified Nash והמדיניות האחרונה נכשלים כבר באיסוף. עלות החישוב נמדדה בנפרד: nash 4.6 עד 19.1 מילישניות, רפליקטור עם רצפה 300 עד 371.



איור 2. exploitability ואנטרופיה לפי סבב לשמונת הפותרים.

7.3 ניסוי 3 — מה יש באוכלוסייה

מתוך 13 מדיניות רק ארבע קיבלו משקל: 0.0361, 0.0799, 0.3322, 0.5518. שלושת האחרונים הם בדיוק פרמטרי שיווי המשקל שסעיף 7.6 מחלץ מההתנהגות, בהתאמה לארבע ספרות. ההסבר מבני: כל פרמטר חופשי נישא במדיניות ודאית אחת, וההסתברות נוצרת מהמשקל שהתערובת נותנת לה. לשחקן הראשון אין מדיניות נשלטת ולשני יש שתיים, מה שמתאים לכך שלשני יש פתרון יחיד. נמצאו עשרה מעגלים בני שלושה, 3.5 אחוז מהשלשות: random מנצח מדיניות אחת, היא מנצחת שנייה, והשנייה מנצחת את random. אין מנצחת יחידה, ולכן הפתרון חייב להיות תערובת. מדד הגיוון צונח מ-0.4175 לאפס בשלושת הסבבים האחרונים, אך אינו מנבא חשיבות: המדיניות המגוונות ביותר לא קיבלה משקל כלל, וזו שקיבלה 0.5518 היא בגיוון אפס. גיוון אפס אינו סימן לעצור.



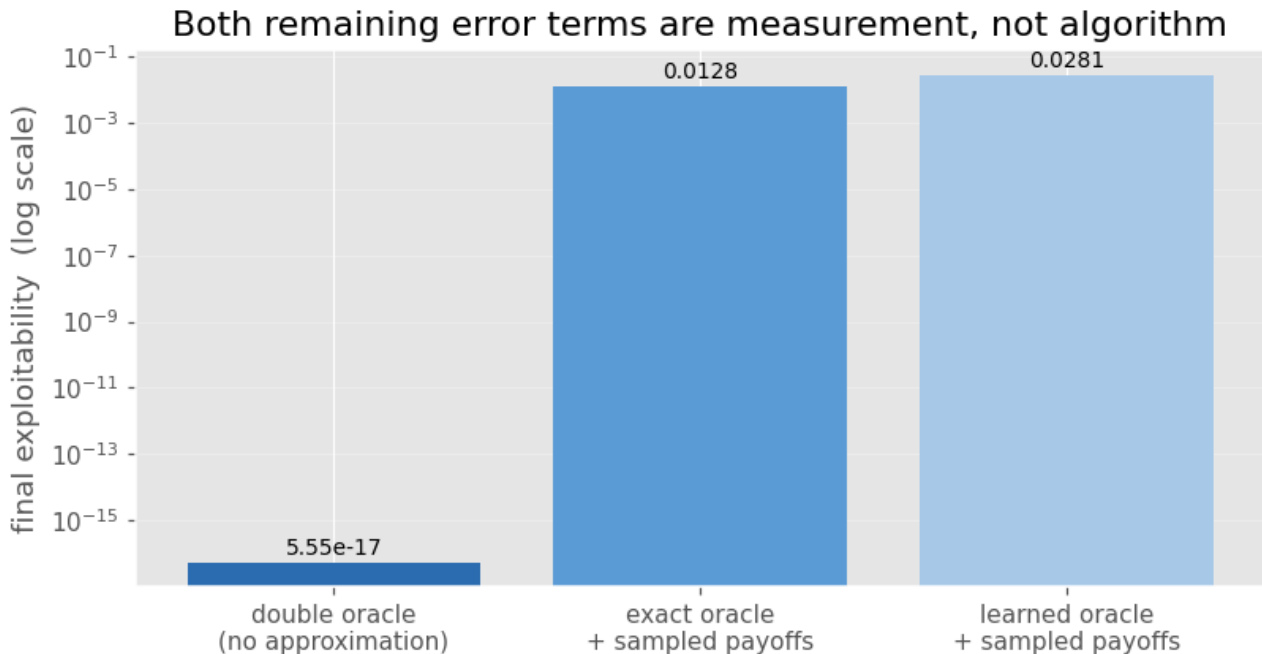
איור 3. טבלת מי-מנצח-את-מי, המשוערת שראה הפותר מול המדויקת. המדיניות שקיבלו משקל מסומנות.

7.4 ניסוי 4 — איכות המאמן, ומאיפה באה השגיאה

ארבעה תקציבי אימון — 50,000, 10,000, 2,000, 500 — אפיוזדות לסבב — נותנים 0.0448, 0.0598, 0.0429, 0.0281. הדירוג אינו עולה בסדר ורווחי הסמך חופפים כולם. מה שכן משתפר בסדר הוא סטיית התקן, מ-0.0360 ל-0.0111: התקציב קונה יציבות, לא איכות ממוצעת.

כדי לדעת מאיפה באה השגיאה כיביתי את שני הקירובים בזה אחר זה. double oracle מלא, בלי אימון ובלי דגימה, מגיע ל- $5.55e-17$ בתוך 0.23 שניות. מאמן מדויק עם טבלה משוערת עוצר ב-0.0128, וזה כל מחיר הדגימה. המאמן הנלמד הטוב ביותר מגיע ל-0.0281, כלומר מוסיף 0.0153.

שני האיברים מאותו סדר גודל, ושניהם שגיאת מדידה ולא שגיאת אלגוריתם. זה מנוגד לאינטואיציה: המאמן בולע 1.2 מיליון אפיזודות ו-60 שניות ובכל זאת אינו הרכיב הדומיננטי. המסקנה המעשית היא שאין טעם להשקיע תקציב רק באחד מהשניים.



איור 4. exploitability סופי בסקאלה לוגריתמית: double oracle, מאמן מדויק עם טבלה משוערת, ומאמן נלמד.

7.5 ניסוי 5 — יריבים שלא נראו באימון

נבחנו סוכני הבסיס, שמונה מדיניות אקראיות מזרע נפרד, שלוש הפרעות של שיווי המשקל, מדיניות מוקדמות של האוכלוסייה, והפתרון המתמטי עצמו.

self_play מרוויח יותר בכל שלושת הממוצעים מול יריבים שאינם שיווי משקל, 0.1333, 0.4347 ו-0.0576, כי מדיניות תוקפנית וודאית גובה מיריב חלש ביד רחבה. אבל אותה תוקפנות מפסידה 1.25 למי שידע לנצל אותה. מול הפתרון המתמטי nash מקבל בדיוק -0.0556, שהוא ערך המשחק, וכל השאר מפסידים יותר. הרגישות של self_play לזהות היריב היא 0.2465 מול 0.0301.

7.6 הראיה ההתנהגותית — הישר של Kuhn

הבדיקה החזקה ביותר אינה מספר אלא התנהגות. מהתערובת שנלמדה חולצו הפרמטרים שהפתרון המתמטי מדבר עליהם: α 0.0361, β 0.3322, γ 0.0799, והמרחק ממשפחת הפתרונות הוא 0.0372 על משפחה שנפרשת על טווח של $1/3$. לשחקן השני התקבלו 0.3648 ו-0.3552, במרחק 0.0533 מהפתרון היחיד שבו שניהם $1/3$.

חמש הפעולות שבהן לשיווי המשקל אין דרגת חופש יצאו מדויקות, והן לא כוונו אלא צצו מהאופטימיזציה. אף אחד מחמשת מודולי האימון אינו מזכיר את הפתרון המתמטי — הוא משמש רק להערכה בדיעבד. ניתוח 5,000 ידיים

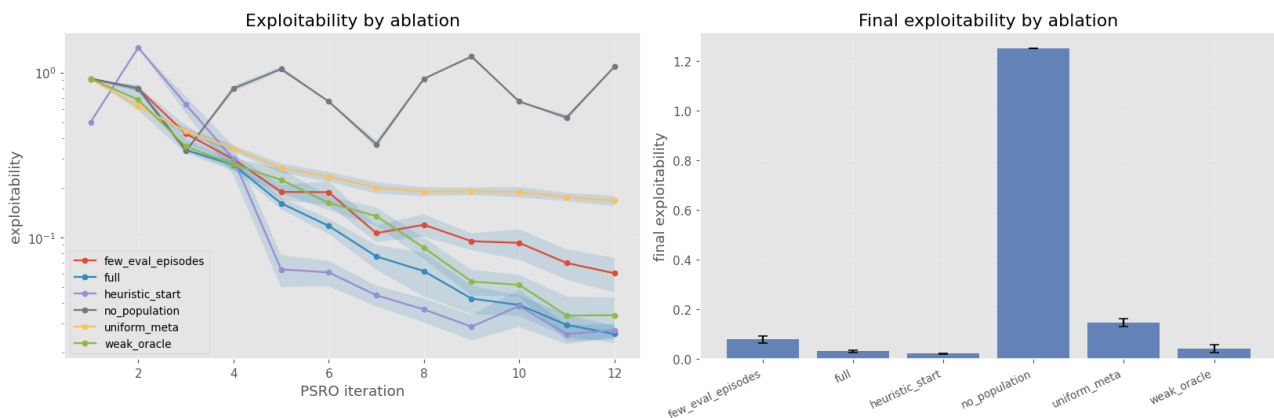
משלים: 84 אחוז מהידיים נגמרות בלי הימור יזום, והבלוף מופיע ב-1.2 אחוז מהן ומפסיד 0.95 ליד, ובכל זאת הכרחי: בלעדיו היריב מקפל מול כל הימור.

8. אבלציות – מה באמת עושה את העבודה

מה שונה	exploitability	רווח 95%	יחס לייחוס	מופרד?
בלי אוכלוסייה	1.2500	0.0000	39.67	כן
תערובת אחידה	0.1462	0.0469	4.64	כן
פחות אפיזודות הערכה	0.0786	0.0387	2.50	על הגבול
מאמן חלש	0.0429	0.0447	1.36	לא
זרוע הייחוס	0.0315	0.0150	1.00	—
התחלה היוריסטית	0.0220	0.0063	0.70	לא

חמש אבלציות, כל אחת משנה רכיב אחד. סדר הגדלים הוא הממצא המעשי, אבל רווחי הסמך מחייבים זהירות: רק שתי המדרגות הראשונות מופרדות סטטיסטית, ההערכה נוגעת בקצה הרווח של הייחוס, והמאמן וההתחלה ההיוריסטית בתוך הרעש.

המסקנה חדה יותר מארבע מדרגות. מה שחשוב הוא קיום האוכלוסייה וחשוב נכון של התערובת, וקיום האוכלוסייה לבדו אינו מספיק: תערובת אחידה על אותה אוכלוסייה מחמירה פי 4.64. תקציב חישוב מוגבל כדאי להשקיע במבנה האלגוריתם ולא בחיזוק הלומד.



איור 5. exploitability לפי סבב וסופי לחמש האבלציות מול זרוע הייחוס.

9. מגבלות

- המשחק קטן, וזה מה שמאפשר את החישוב המדויק. בסביבות גדולות ה-exploitability עצמו נאמד בקירוב ומהווה חסם תחתון בלבד.
- מספר הסבבים חסום ב-12, וחמישה seeds מפרידים את self-play מ-PSRO אך לא את nash מ-fictitious_play (0.0315 מול 0.0282).
- ההתכנסות המהירה היא תכונה של Kuhn ולא הבטחה כללית. Zhang ו-Sandholm (IJCAI 2024) בנו משחקים שבהם double oracle דורש מספר סבבים מעריכי, וגם כאן ה-exploitability עולה בחלק מהסבבים.

- סף העצירה נקבע לאפס בכוונה, כדי שכל הזרועות ירוצו אותם 12 סבבים. זמן הריצה נמדד תחת הרצה מקבילית ולכן מזהם, ומשום כך עלות הפותר נמדדה בנפרד.

10. שיפורים אפשריים

הרחבה ל-Leduc Poker, שבו נדרש exploitability מקורב; PSRO למשחקים שאינם סכום אפס; Anytime PSRO, שפותר במפורש את חוסר המונוטוניות שנמדדה כאן; רפליקטור בממוצע לאורך זמן; ויותר seeds להפרדת שלושת הפותרים הראשונים.

11. הרחבת בונס

ההרחבה המוצהרת היא Rectified PSRO של Balduzzi ואחרים (ICML 2019): כל לומד מתאמן רק נגד יריבים שהוא אינו מפסיד להם, כדי לייצר התמחויות. הפותר מומש, נכלל בהשוואה, והכשל המתועד שלו שוחזר על מימוש עצמאי — exploitability של 0.5178, ורק 5.4 מדיניות שונות מתוך 13 שהוסיף, כלומר הוא מייצר עותקים.

זה תואם את ההוכחה של McAleer ואחרים (NeurIPS 2020) שהשיטה אינה מתכנסת תמיד, ואת הלולאה ש-Muller ואחרים (ICLR 2020) מצאו ב-Kuhn עצמו. לזכות המאמר: השיטה פותחה למשחקים סימטריים ו-Kuhn אינו כזה, ולכן זו הדגמה של גבול התחולה ולא הפרכה. כתוספת נעשו מבחנים סטטיסטיים על חמישה seeds.

12. מסקנה

התשובה לשאלה המרכזית היא כן, ובאופן חד. באותו תקציב אימון PSRO מגיע ל-exploitability של 0.0315 מול 1.2500 ב-self-play, פי 39.7, בלי חפיפה על פני חמישה seeds. הוא מתכנס לערך המשחק בשגיאה 0.0004 מול 0.8056 ואינו מסתובב במעגל. האוכלוסייה שלו מכילה עשרה מעגלים לא-טרנזיטיביים ולכן הפתרון חייב להיות תערובת, והתערובת שנמצאה יושבת על משפחת הפתרונות של Kuhn במרחק 0.0372 — בלי ששום דבר בקוד רמז על קיומה. המחיר: מול יריבים חלשים self-play מרוויח יותר. עמידות ויכולת ניצול אינן אותו דבר. והאבולוציות אומרות מה עושה את העבודה — האוכלוסייה וחישוב התערובת, ולא חוזק הלומד.

13. חלוקת עבודה

הפרויקט בוצע במלואו, ביחידות, על ידי נדב פיירמן: הסביבה, החישוב המדויק, המטא-משחק, המאמנים, הפותרים, לולאת PSRO, הניסויים, הבדיקות, המחברת, הדוח והמצגת.

נספח — קבצים ותוצרים

- שישה קובצי configs — הפרמטרים של כל ריצה: פותר, מאמן, seed ותקציבים.
- שישה קובצי csv — מדדים לפי סבב: exploitability, אנטרופיה, ערך, אוכלוסייה, אפיזודות וזמן.
- חמישה קובצי סיכום — שורה לכל ריצה עם המדדים הסופיים; לניסוי 3, שרץ על seed יחיד, אין סיכום.
- שישה קובצי json — הטבלאות המשוערות והמדויקות, התערובות והתמיכה; 1,004 קילובייט, רוב הנפח. בסך הכול 23 קבצים ו-1,460 קילובייט, לצד תשעת קובצי הקוד, המחברת (128 תאים, 36 מהם קוד), וקובצי README ו-requirements להתקנה ולהרצה.